# Token-ordered LRU page replacement: a simulator-based case study

Technical report on the implementation and evaluation of the algorithm of S. Jiang and X. Zhang, *Token-ordered LRU: an effective page replacement policy and its implementation in Linux systems*, Performance Evaluation 60 (2005) 5–29.

> **Scope statement.** This is a **discrete-event simulator** reproduction. It is not a Linux kernel modification, and all workloads are **synthetic** approximations of the paper's programs (names carry a * in all figures), parameterised from the paper's Table 1 and Figs. 1–5 and scaled to 1/8. Time is simulated ticks, never wall-clock. Published numbers are shown only as labelled reference data.

## 1. Summary

We implemented a conventional global-LRU baseline and the token-ordered LRU policy (paper Sections 5.2–5.3, including the synchronisation-aware priority) in one simulator, ran both on identical workload pairs mirroring the paper's five interacting groups plus the Section 5.4 example, and measured page faults, slowdown, RSS/MAD/NAP, token activity and false-LRU evictions over 5 seeds, a memory-pressure sweep and ablations.

- **Mechanism confirmed.** Counted directly, evictions of pages belonging to a process that was itself faulting ("false LRU" evictions, paper Sec. 1) fell from 197,434 (cross-process) + 124,746 (self) under global LRU to 21,095 + 35,187 under token-ordered LRU, summed over the five pairs and 5 seeds.
- **Clear wins** in bit-r (faults −98%, slowdown −51%), LU-1/LU-2 (slowdown 42.4 → 2.8 / 5.4), vortex3 in vortex3/gcc, and the Section 5.4 example (total faults 316,400 → 8,145).
- **Unfavourable or neutral cases are reported, not hidden:** gzip is unchanged; gcc is much worse in bit-r/gcc (faults 210 → 5,021) and the *pair-average* slowdown there rises 2.01 → 2.17; vortex1 gets worse on average in vortex1/vortex3; at low shortage the policy is neutral or slightly worse.
- **Synchronisation-aware priority (Sec. 5.3) is implemented** and evaluated on a bulk-synchronous scenario of our own design (Section 6.7): under the paper's default token handling it never acts; where the token is re-assigned while processes wait at barriers it helps in most runs (40 of 72 paired seed runs better than random assignment, 9 worse, 23 identical), with clear counter-examples.
- **Agreement with the paper is partial:** same sign of improvement for 6/10 programs (faults) and 7/10 (slowdown). Absolute magnitudes are not comparable.

## 2. Background: false LRU pages and the token

Global LRU treats a page as replaceable when it has not been touched recently. A process blocked on a page fault cannot touch its pages, so they age and are taken by other processes ("false LRU" pages), which then fault in turn; the effect cascades into thrashing (paper Sec. 1, 4.3). Token-ordered LRU gives one process at a time a temporary privilege: while it holds the token *and* is swapping, its pages are skipped by victim selection so it can build its working set; when its swapping status turns off the privilege lapses, so its true-LRU pages can be replaced again. Two monitors (Sec. 5.3) retire an abusive holder.

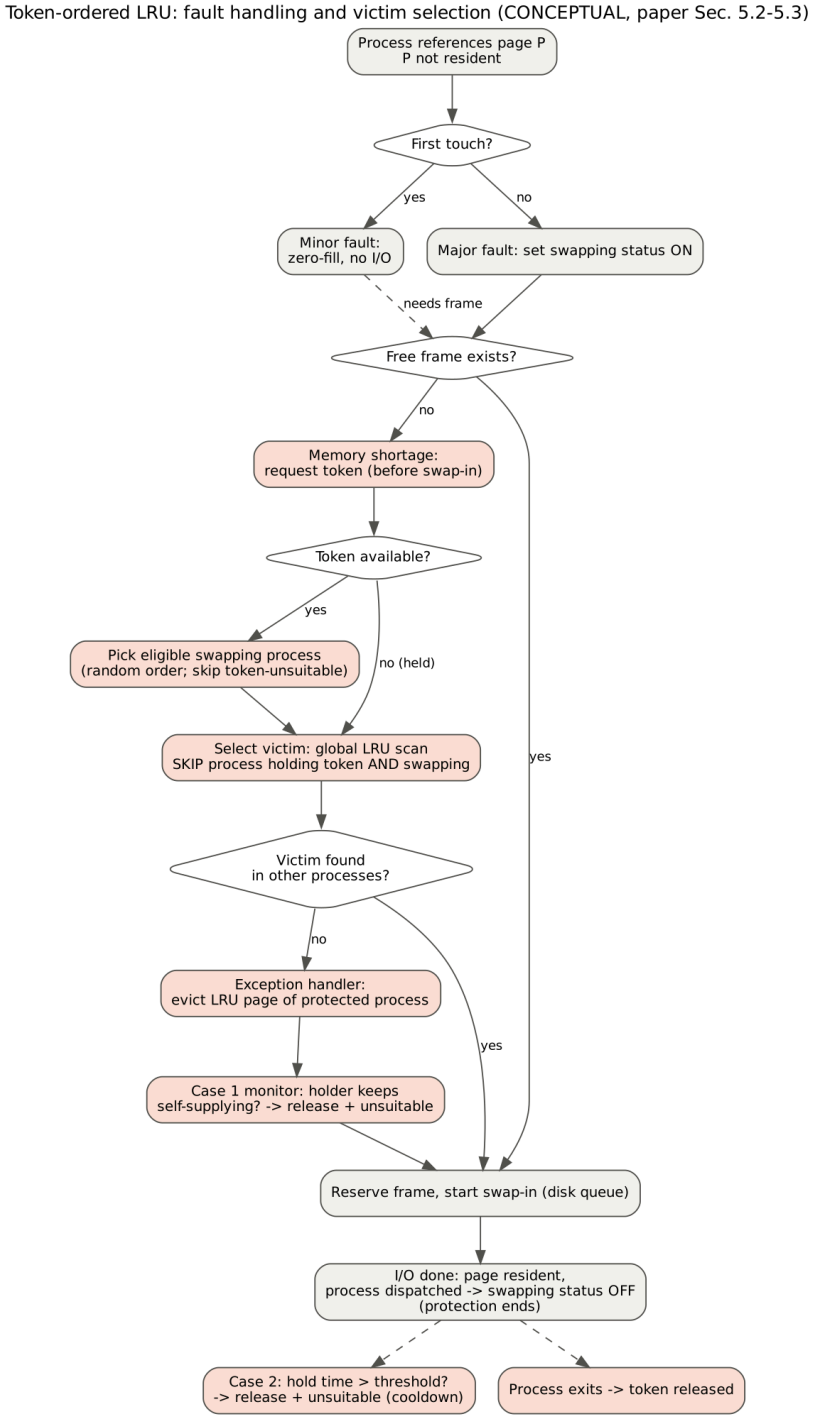



Figure A (conceptual illustration, not measured data): fault handling and victim selection in token-ordered LRU.

# 3. Implementation and paper mapping

| Paper element | Code | Status |
|---|---|---|
| Global, mutually exclusive token (available / held) | TokenOrderedLRU.owner, _acquire, _release | Implemented |
| Token requested only on a page fault caused by memory shortage, before the swap-in (do_swap_page) | Simulator._fault → policy.on_major_fault(shortage = no free frame) | Implemented |
| Per-process swapping status; on while swapping, off on return to normal execution | Process.swapping; set on major fault, cleared on next dispatch | Implemented (simulator interpretation) |
| __get_free_pages() skips the program holding both token and swapping status | TokenOrderedLRU.protected() and select_victim() | Implemented |

| Paper element | Code | Status |
|---|---|---|
| Protection ends as soon as swapping ends | protected() re-evaluated on every victim search | Implemented |
| Exception handler: take the holder's own LRU pages if no one else can supply | fallback branch of select_victim() | Implemented |
| Case 1: holder repeatedly selects pages from itself → release + token-unsuitable | _record_holder_fault (window / ratio) | Implemented (thresholds ours) |
| Case 2: holding time over threshold → release + token-unsuitable for a period | on_tick, unsuitable_until | Implemented (threshold ours) |
| Random token assignment among interacting programs | _choose_acquirer (seeded RNG) | Implemented |
| Priority for processes others are waiting on (synchronisation / blocking messages) | Workload.sync_group/barriers, Simulator.waiters_on(), TokenOrderedLRU._choose_acquirer | Implemented for barrier-synchronised processes on one node; remote nodes and message passing are not modelled |
| (Extension, not in the paper) hand a held token to a waited-on process | cfg.sync_transfer (default off) | Our addition, evaluated separately |
| Linux 2.2.14 kernel implementation | — | NOT implemented (simulator only) |

**Execution model.** One CPU with round-robin scheduling (quantum 50 references); a hit costs 1 tick; one shared FIFO disk services each swap-in in 100 ticks; frames are reserved when a swap-in starts; evictions are free (no write-back). **Swapping status** is turned on when a major fault starts and off when the process is next dispatched, i.e. across the disk wait *and* the ready-queue wait. The token is kept until the owner exits or a monitor forces a release, as the paper's description implies.

**Fault terminology.** The paper counts only *major* faults (page fetched from disk). In this simulator "minor" means first-touch zero-fill (no I/O), which is a different use of the word than the paper's (page-table relink of a resident page); all fault counts and reductions here are major faults only.

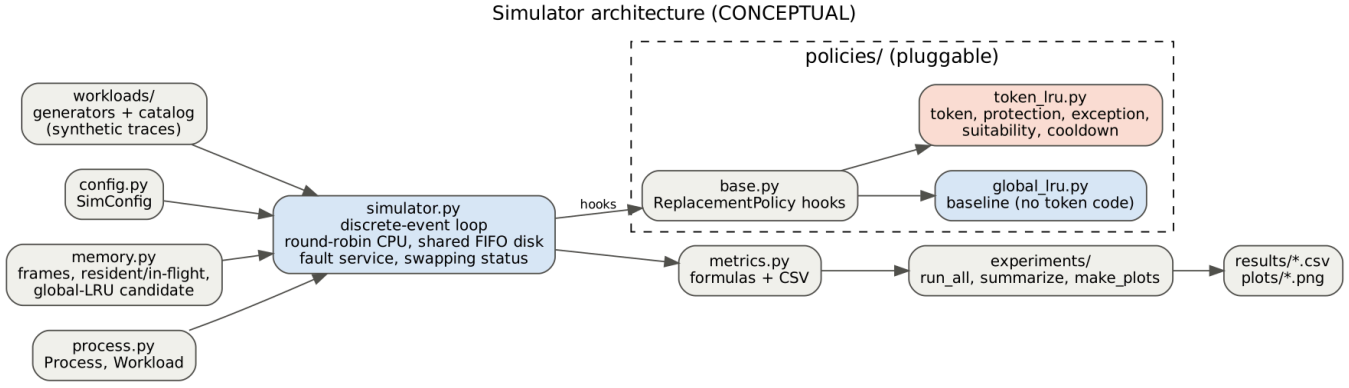



Figure B (conceptual illustration): simulator architecture. The baseline policy contains no token code.

## 4. Workloads: audit against the paper

An earlier version of this project took its workload parameters from memory. They were re-audited against Table 1, Figs. 1–8 and the Section 5.4 text and corrected where they disagreed; all results in this report come from the corrected workloads.

| Program | What the paper states | What the synthetic workload uses | Status |
|---|---|---|---|
| gzip | Table 1: 197.4 MB, 248.7 s; Fig. 2: MAD ≈49.5k pages from start, NAP ≈1k with sweep spikes at ≈42/50/90/97/148/150/198/200/245 s | 49,500 pages; hot set 1,000; 9 sweep events at those times/sizes; starts ≈250 s into the pair (Fig. 6) | Corrected (was 6 sweeps) |
| vortex1 / vortex3 | Table 1: 115.0 / 131.2 MB, 342.3 / 398.0 s; Fig. 3: stair-climbing MAD, RSS ≈ 57–80% of MAD | 29,400 / 33,600 pages, 13 / 14 stairs, working set 60% of peak MAD | Corrected (WS was 75%) |

| Program | What the paper states | What the synthetic workload uses | Status |
|---|---|---|---|
| bit-r | Table 1: 131.3 MB, 326.1 s; Fig. 3: RSS ≈33k for 0–168 s then ≈1.8k (plotted MAD ≈65.5k, inconsistent with Table 1) | 33,600 pages cyclic for 168 s, then 1,800-page hot set for 158 s | Corrected (switch was 165 s); MAD discrepancy noted |
| gcc | Table 1: 145.0 MB max, 218.7 s; Fig. 1: ≈3–4k to 75 s, spike ≈33.5k 75–85 s, ≈12.5k to 108 s, ≈5k to 182 s, spike ≈36.5k 182–200 s, ≈13k to 218.7 s; Fig. 8: starts ≈250 s in vortex3/gcc | Phases 3.5k/4k/33.5k/12.5k/5k/36.5k/13k with those durations; memory is freed between phases | Corrected (durations and frees) |
| LU | Table 1: 161.2 MB, 98.0 s; Fig. 5: MAD ≈40.3k, matrix released and re-allocated at ≈0/21/41/61/81 s, NAP ≈2.2k pages/s between | 40,300 pages; 5 iterations of free + re-allocate + slow sequential traversal of the whole matrix; LU-1 = LU-2 (same program, footnote 4) | Corrected twice: (1) added free/re-allocate; (2) hot set replaced by whole-matrix traversal |
| Section 5.4 program | Sec. 5.4 code: 1000 loops, SIZE ×0.9 every 10 loops; 53 MB and 58 MB arrays, 60 MB memory | Same loop structure, scaled 1/4 (3,392 and 3,712 pages in 3,840 frames) | Verified |

# 5. Experimental design

- Both policies run on identical traces, capacity, I/O model and start offsets; token assignment uses a seeded RNG; 5 seeds (0–4).
- Slowdown = interacting elapsed time / elapsed time of the program alone with no major faults (paper Sec. 4.1). Reductions use the formulas of Section 6 of the brief.
- Memory shortage = 1 − capacity / Σ peak MAD (paper footnote 5). For each pair the shortage in the 20–50% range was chosen **from baseline runs only**: the smallest value at which the baseline's slowdown exceeds 1.5× its CPU-sharing floor. The token policy never influenced this choice.
- Thresholds not given by the paper are our choices: Case 2 hold limit = 4× the longest dedicated run, same cooldown; Case 1 window 64 faults with ≥90% self-supplied; I/O time 100 ticks; quantum 50.

| Pair | Shortage | Baseline slowdown / CPU-sharing floor (max over programs) | Chosen |
|---|---|---|---|
| gzip_vortex3 | 20% | 1.19 | |
| gzip_vortex3 | 30% | 1.42 | |
| gzip_vortex3 | 40% | 1.53 | ✔ |
| gzip_vortex3 | 50% | 1.46 | |
| bitr_gcc | 20% | 1.26 | |
| bitr_gcc | 30% | 1.38 | |
| bitr_gcc | 40% | 1.51 | ✔ |
| bitr_gcc | 50% | 3.24 | |
| vortex3_gcc | 20% | 1.00 | |
| vortex3_gcc | 30% | 1.06 | |
| vortex3_gcc | 40% | 1.72 | ✔ |
| vortex3_gcc | 50% | 2.45 | |
| vortex1_vortex3 | 20% | 1.00 | |
| vortex1_vortex3 | 30% | 1.00 | |
| vortex1_vortex3 | 40% | 1.00 | |
| vortex1_vortex3 | 50% | 1.08 | ✔ |
| lu1_lu2 | 20% | 21.21 | ✔ |
| lu1_lu2 | 30% | 21.21 | |
| lu1_lu2 | 40% | 21.21 | |
| lu1_lu2 | 50% | 21.21 | |

# 6. Results

## 6.1 Per-program results (mean of 5 seeds)

| Pair | Program | Shortage | LRU faults | Token faults | Fault red. | LRU slowdown | Token slowdown | Slowdown red. | Seeds better (of 5) |
|---|---|---|---|---|---|---|---|---|---|
| gzip / vortex3 | gzip* | 40% | 3,938 | 3,938 | +0% | 2.43 | 2.44 | -1% | 0 |
| gzip / vortex3 | vortex3* | 40% | 229 | 158 | +31% | 1.15 | 1.14 | +1% | 5 |
| bit-r / gcc | bit-r* | 40% | 5,310 | 83 | +98% | 2.57 | 1.27 | +51% | 5 |
| bit-r / gcc | gcc* | 40% | 210 | 5,021 | -2289% | 1.45 | 3.07 | -112% | 0 |
| vortex3 / gcc | vortex3* | 40% | 1,102 | 172 | +84% | 1.62 | 1.25 | +22% | 5 |
| vortex3 / gcc | gcc* | 40% | 3,129 | 2,576 | +18% | 2.77 | 2.28 | +18% | 5 |
| vortex1 / vortex3 | vortex1* | 50% | 649 | 947 | -46% | 2.03 | 2.08 | -2% | 3 |
| vortex1 / vortex3 | vortex3* | 50% | 1,015 | 708 | +30% | 1.96 | 1.88 | +4% | 4 |
| LU-1 / LU-2 | LU-1* | 20% | 25,190 | 1,063 | +96% | 42.41 | 2.79 | +93% | 5 |
| LU-1 / LU-2 | LU-2* | 20% | 25,190 | 4,205 | +83% | 42.41 | 5.35 | +87% | 5 |

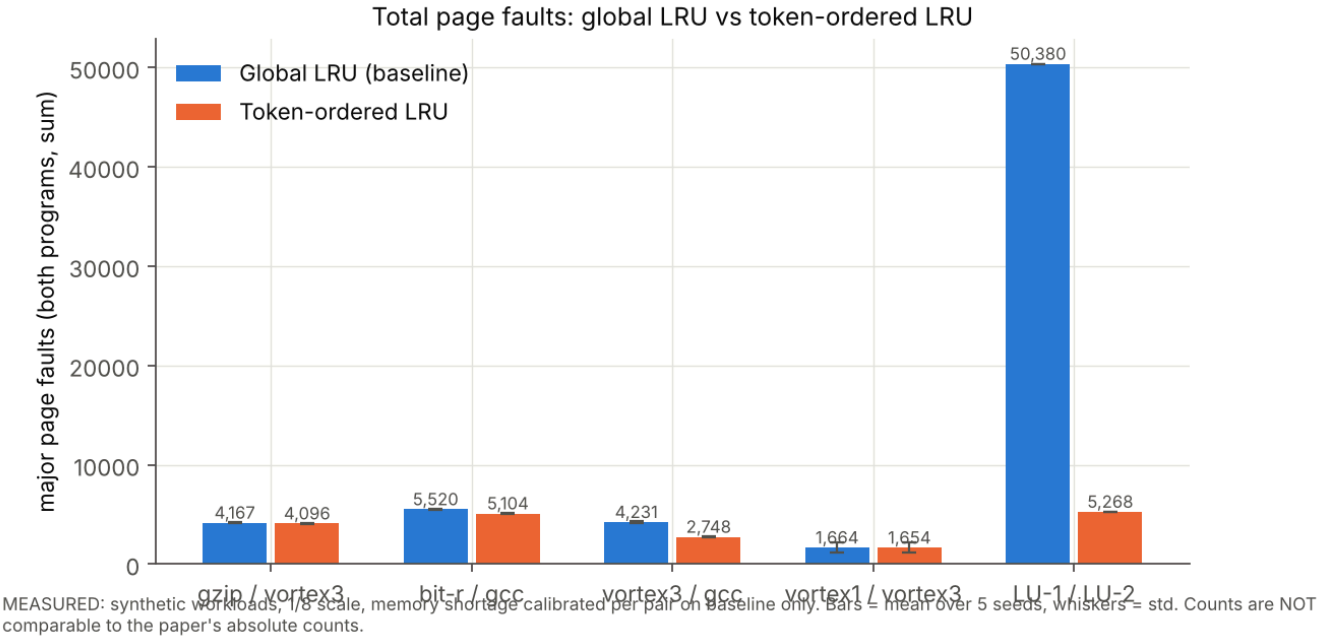



Figure 1 (measured): total major faults per pair. Bars = mean of 5 seeds; whiskers = std.

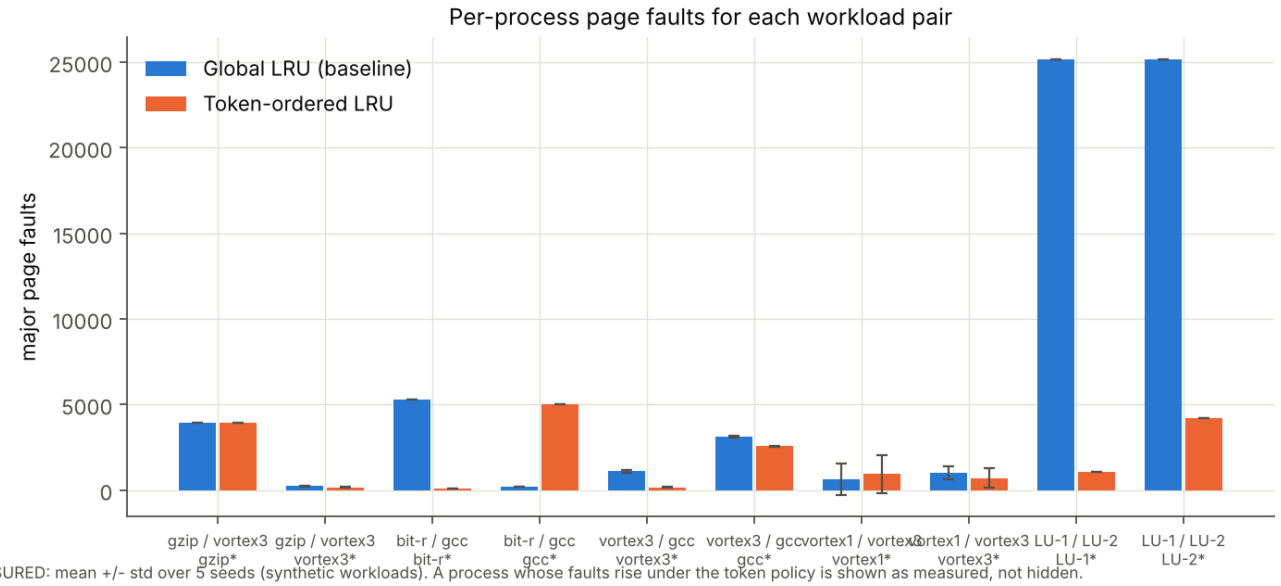



Figure 2 (measured): major faults per process. Processes that get worse are shown as measured.

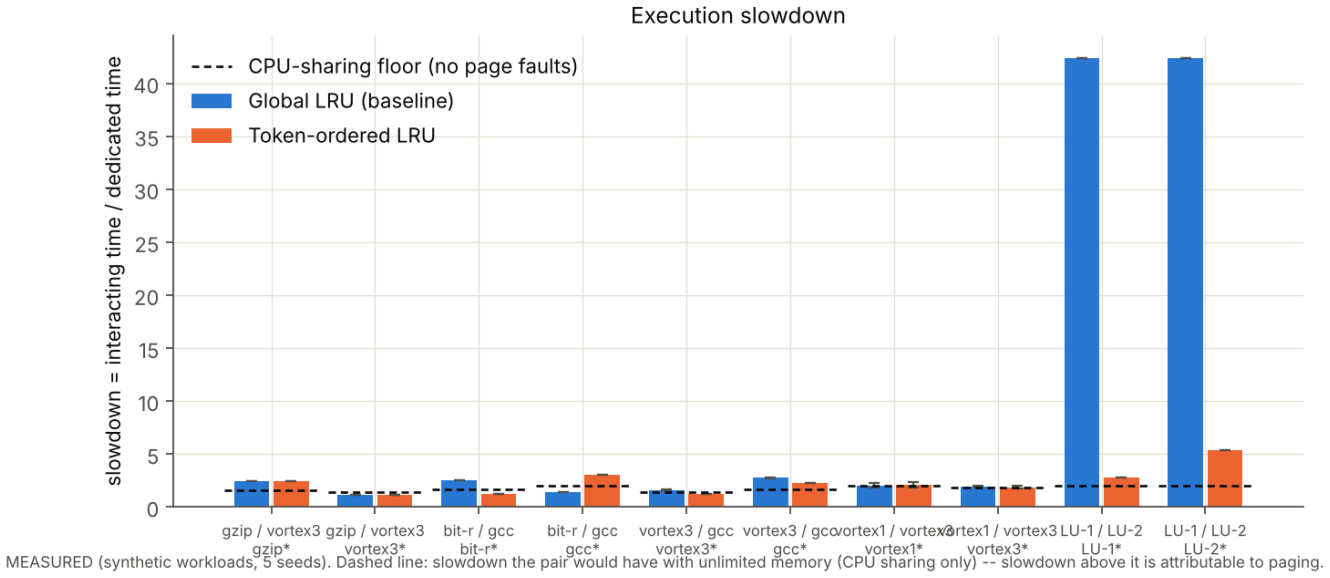
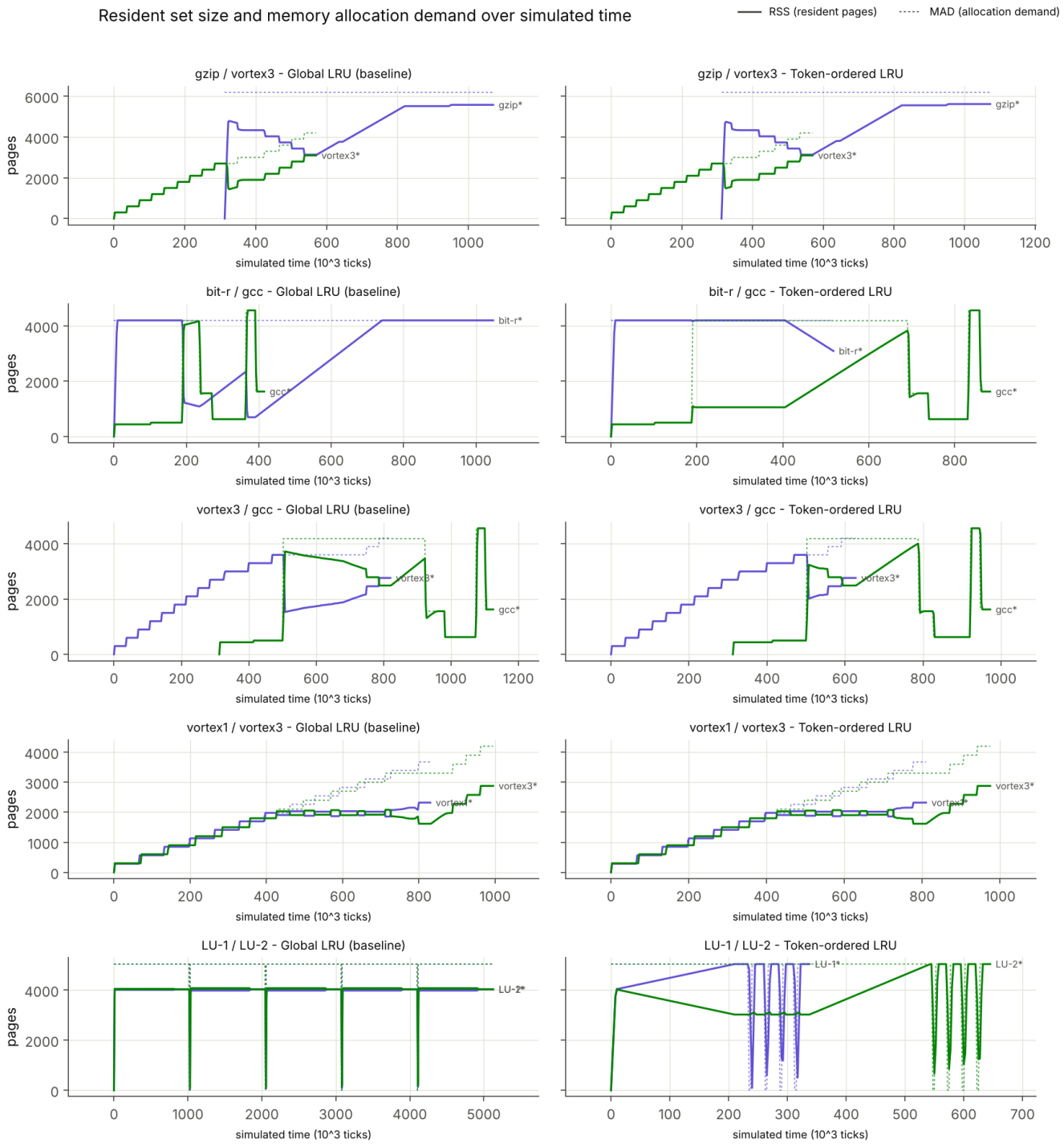


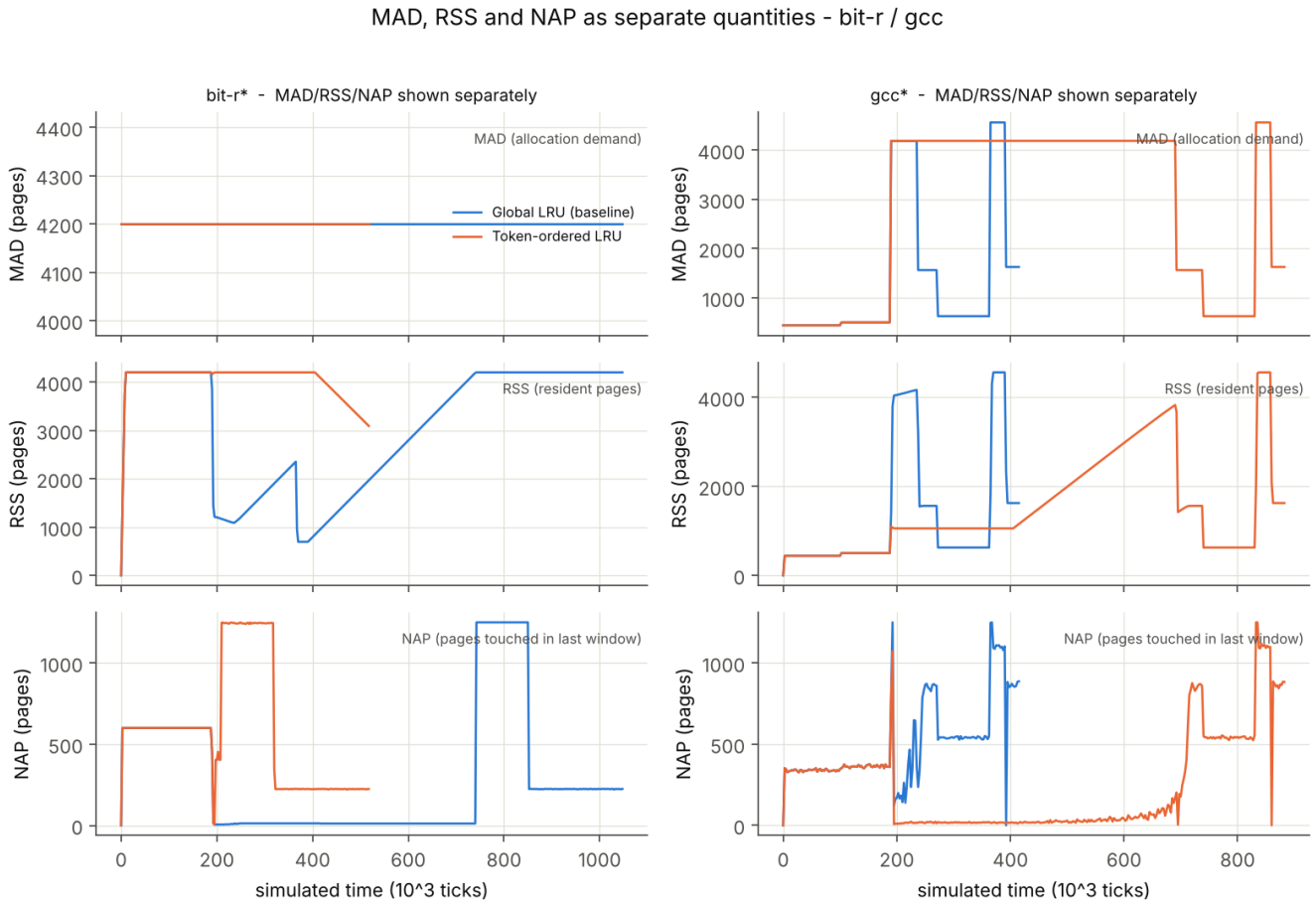

Figure 3 (measured): slowdown; dashed line = slowdown from CPU sharing alone (no paging).

## 6.2 Memory behaviour: RSS, MAD and NAP




MEASURED, seed 0, synthetic workloads. Compare the RSS of each process between the two columns (the paper's Figs. 6-10 vs 12-16 analogue); time axes differ because completion times differ.

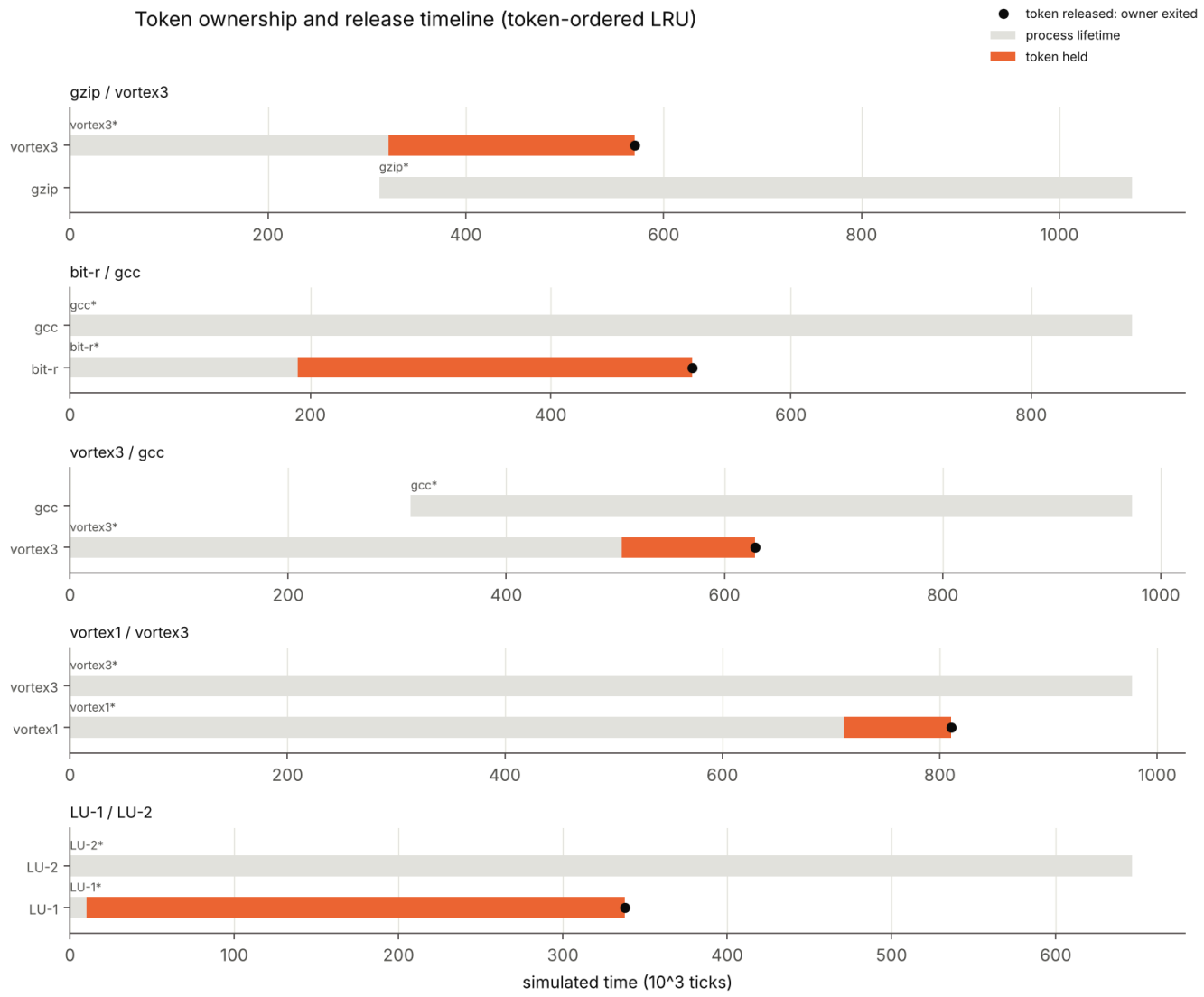
Figure 4 (measured, seed 0): RSS (solid) and MAD (dashed) over simulated time, baseline (left) vs token-ordered LRU (right).




MEASURED, seed 0, synthetic workloads. NAP window = 1 scaled paper-second. MAD is defined by the workload generator; RSS and NAP are measured by the simulator.

Figure 5 (measured, seed 0): MAD, RSS and NAP shown as separate quantities for bit-r/gcc.

## 6.3 Token behaviour



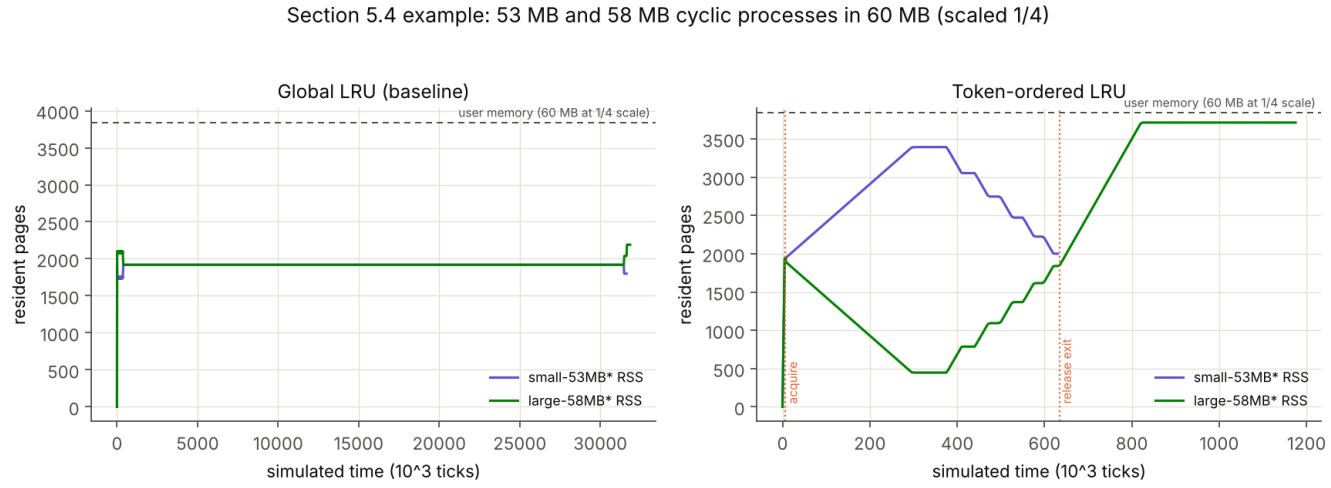

MEASURED, seed 0, synthetic workloads: orange = interval during which the process owns the global token.

Figure 6 (measured, seed 0): token ownership intervals. The token is held from the first shortage-triggered fault until the owner exits.

| Policy | False-LRU evictions by OTHER processes | False-LRU evictions by own fault | Token acquisitions | Protected skips | Fallback evictions | Forced releases (timeout) | Forced releases (self-supply) |
|---|---|---|---|---|---|---|---|
| global_lru | 197,434 | 124,746 | 0 | 0 | 0 | 0 | 0 |
| token_lru | 21,095 | 35,187 | 25 | 42,278 | 0 | 0 | 0 |

In the main runs no forced releases occurred (the 4× hold threshold was never exceeded and the Case 1 ratio was never reached), and the fallback exception handler was never needed. Protection did act: 42,278 victim searches would have chosen the protected process under plain LRU. The ablation below shows the Case 2 threshold *does* matter when set tightly.

### 6.4 Section 5.4 example (53 MB and 58 MB arrays, 60 MB memory, scaled 1/4)



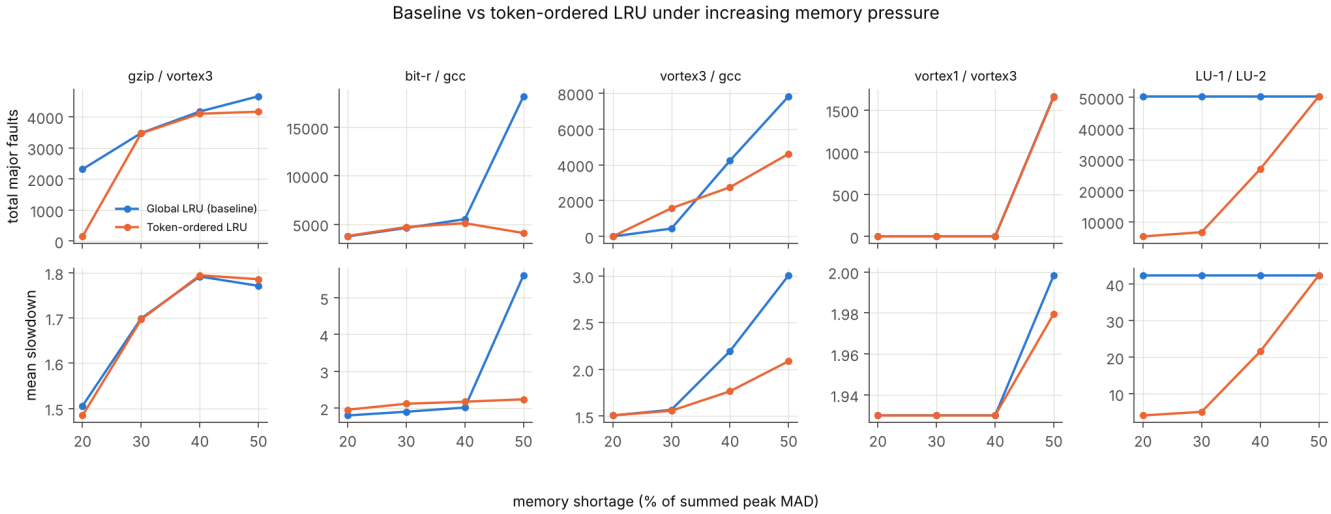

MEASURED, synthetic re-implementation of the paper's program. Baseline = exact LRU on a cyclic scan (its worst case), so the baseline slowdown here is far larger than the paper's Linux measurement.

Figure 7 (measured): RSS of the two processes. The token goes to the small process; the large one shrinks, then grows once the small one reduces its working set, as the paper describes for its Fig. 11.

| Policy | Process | Major faults | Elapsed (ticks) | Dedicated (ticks) | Slowdown |
|---|---|---|---|---|---|
| global_lru | small-53MB* | 157,248 | 31,631,662 | 336,360 | 94.04 |
| global_lru | large-58MB* | 159,152 | 31,852,098 | 368,220 | 86.50 |
| token_lru | small-53MB* | 1,481 | 634,029 | 336,360 | 1.88 |
| token_lru | large-58MB* | 6,664 | 1,179,086 | 368,220 | 3.20 |

Baseline slowdown is extreme because exact LRU on a cyclic scan larger than memory faults on every reference; the paper's Linux baseline was less pathological, so the magnitude should not be compared with the paper.

## 6.5 Memory-pressure sweep (20–50% shortage)




MEASURED, mean over 5 seeds, synthetic workloads. Shortage = 1 - capacity / sum of the programs' peak MAD. The paper's 20-50% range refers to ITS machine settings; here it is an explicit sweep parameter.

Figure 8 (measured): effect of memory shortage. The paper’s 20–50% is a property of its machine settings; here it is an explicit sweep parameter.

| Pair | Shortage | LRU faults | Token faults | Fault red. | LRU mean slowdown | Token mean slowdown | Slowdown red. | Seeds better (of 5) |
|---|---|---|---|---|---|---|---|---|
| bitr_gcc | 20% | 3,736 | 3,784 | -1% | 1.80 | 1.95 | -9% | 0 |
| bitr_gcc | 30% | 4,610 | 4,710 | -2% | 1.90 | 2.12 | -11% | 0 |
| bitr_gcc | 40% | 5,520 | 5,104 | +8% | 2.01 | 2.17 | -8% | 0 |
| bitr_gcc | 50% | 18,209 | 4,084 | +78% | 5.61 | 2.24 | +60% | 5 |

| Pair | Shortage | LRU faults | Token faults | Fault red. | LRU mean slowdown | Token mean slowdown | Slowdown red. | Seeds better (of 5) |
|---|---|---|---|---|---|---|---|---|
| gzip_vortex3 | 20% | 2,313 | 152 | +93% | 1.51 | 1.48 | +1% | 5 |
| gzip_vortex3 | 30% | 3,476 | 3,467 | +0% | 1.70 | 1.70 | +0% | 5 |
| gzip_vortex3 | 40% | 4,167 | 4,096 | +2% | 1.79 | 1.79 | -0% | 1 |
| gzip_vortex3 | 50% | 4,658 | 4,161 | +11% | 1.77 | 1.79 | -1% | 1 |
| lu1_lu2 | 20% | 50,380 | 5,268 | +90% | 42.41 | 4.07 | +90% | 5 |
| lu1_lu2 | 30% | 50,380 | 6,633 | +87% | 42.41 | 5.06 | +88% | 5 |
| lu1_lu2 | 40% | 50,380 | 26,944 | +47% | 42.41 | 21.57 | +49% | 5 |
| lu1_lu2 | 50% | 50,380 | 50,380 | +0% | 42.41 | 42.41 | +0% | 0 |
| vortex1_vortex3 | 20% | 0 | 0 | n/a | 1.93 | 1.93 | +0% | 0 |
| vortex1_vortex3 | 30% | 0 | 0 | n/a | 1.93 | 1.93 | +0% | 0 |
| vortex1_vortex3 | 40% | 0 | 0 | n/a | 1.93 | 1.93 | +0% | 0 |
| vortex1_vortex3 | 50% | 1,664 | 1,654 | +1% | 2.00 | 1.98 | +1% | 3 |
| vortex3_gcc | 20% | 0 | 0 | n/a | 1.51 | 1.51 | +0% | 0 |
| vortex3_gcc | 30% | 437 | 1,574 | -260% | 1.57 | 1.56 | +1% | 5 |
| vortex3_gcc | 40% | 4,231 | 2,748 | +35% | 2.20 | 1.77 | +20% | 5 |
| vortex3_gcc | 50% | 7,824 | 4,611 | +41% | 3.01 | 2.09 | +31% | 5 |

The benefit is not uniform: bit-r/gcc is neutral or slightly worse below 50% shortage; LU benefits strongly at 20–40% but not at 50%, where both processes thrash regardless; vortex1/vortex3 has almost no baseline faults below 50%.

### 6.6 Ablations (token policy only; mean slowdown)

| pair | assignment=first_fault | default (hold=4x, random) | hold_factor=2.0 | hold_factor=4.0 | hold_factor=8.0 | hold_factor=off |
|---|---|---|---|---|---|---|
| bitr_gcc | 2.17 | 2.17 | 2.17 | 2.17 | 2.17 | 2.17 |
| gzip_vortex3 | 1.79 | 1.79 | 1.79 | 1.79 | 1.79 | 1.79 |
| lu1_lu2 | 4.07 | 4.07 | 32.19 | 4.07 | 4.07 | 4.07 |
| vortex1_vortex3 | 1.98 | 1.98 | 1.98 | 1.98 | 1.98 | 1.98 |
| vortex3_gcc | 1.77 | 1.77 | 1.77 | 1.77 | 1.77 | 1.77 |

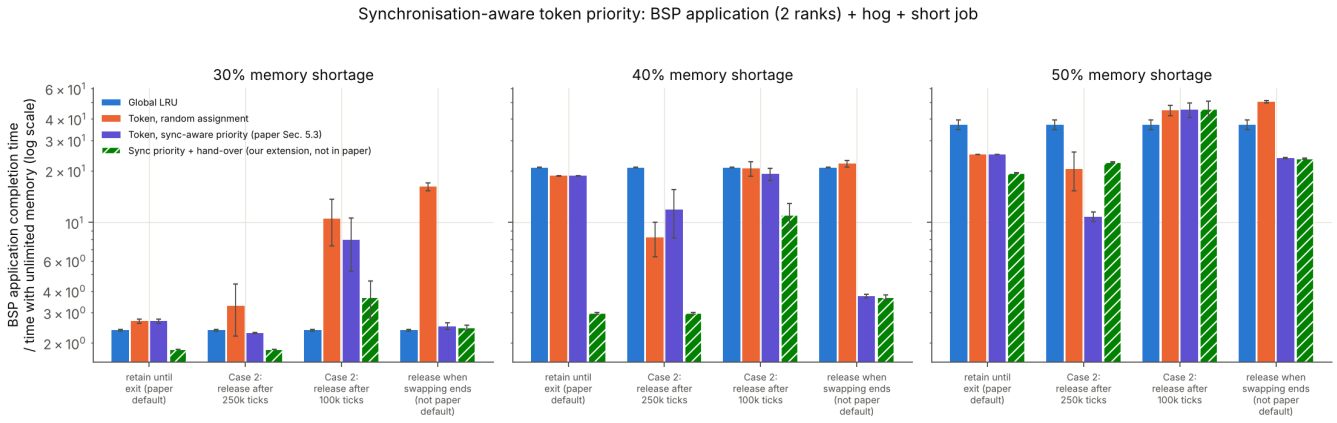
Only LU is sensitive: a Case 2 threshold of 2× the dedicated run forces repeated releases (mean slowdown 32.2 versus 4.1), confirming the paper's warning that the threshold must be large to avoid excessive token transfers. Random versus first-fault assignment made no difference in these runs.

### 6.7 Synchronisation-aware token priority (paper Sec. 5.3)

**What the paper says.** In a parallel system the highest priority for the token goes to a process that others are waiting on (synchronisation or blocking messages), because relieving it benefits the processes coordinating with it; otherwise the token goes to interacting programs in random order.

**How it is modelled.** Processes may belong to a bulk-synchronous group with barrier positions in their traces. A process reaching its barrier blocks until every other live member of its group has reached its own matching barrier. A process is "waited on" when it has not reached the barrier at which at least one peer is blocked; among several candidates the one with most waiters wins, ties are random. The fairness rule still applies (a token-unsuitable process is not eligible). Remote nodes and message passing are not modelled; this is a single-node approximation of the rule. We also built an *extension that is not in the paper* (`sync_transfer`, off by default) that hands an already-held token to a waited-on process.

**Scenario (our design; the paper evaluates none).** Two ranks of a BSP application (10 supersteps, 3 passes over 1,500 / 2,000 pages, barrier after every superstep), an independent memory hog (2,400 pages, cyclic) and a short independent job, with 30/40/50% memory shortage, 6 seeds. Metric: completion time of the BSP application relative to a run with unlimited memory.




MEASURED, 6 seeds, synthetic scenario of our own design (the paper evaluates no BSP scenario). Under the paper-default regime the priority never acts (identical to random); it matters when the token is re-assigned while ranks wait.

Figure 10 (measured): BSP application completion time (log scale) for four token-handling regimes. The priority rule only acts when the token is re-assigned while ranks wait.

| Token handling | Shortage | Global LRU | Token, random | Token, sync priority | Sync priority + hand-over (extension) | Priority assignments (mean) | Seeds sync better | Seeds sync worse | Seeds identical |
|---|---|---|---|---|---|---|---|---|---|
| Case 2 forced release after 100k ticks | 30% | 2.37 | 10.50 | 7.91 | 3.68 | 7.50 | 3 | 1 | 2 |
| Case 2 forced release after 100k ticks | 40% | 20.81 | 20.58 | 19.16 | 11.08 | 16.50 | 5 | 1 | 0 |
| Case 2 forced release after 100k ticks | 50% | 37.11 | 44.82 | 45.20 | 45.69 | 19.00 | 3 | 3 | 0 |
| Case 2 forced release after 250k ticks | 30% | 2.37 | 3.29 | 2.28 | 1.83 | 1.00 | 3 | 0 | 3 |
| Case 2 forced release after 250k ticks | 40% | 20.81 | 8.17 | 11.87 | 2.97 | 4.67 | 2 | 4 | 0 |
| Case 2 forced release after 250k ticks | 50% | 37.11 | 20.45 | 10.82 | 22.46 | 5.33 | 6 | 0 | 0 |
| release when swapping ends (not paper default) | 30% | 2.37 | 16.21 | 2.50 | 2.45 | 894.67 | 6 | 0 | 0 |
| release when swapping ends (not paper default) | 40% | 20.81 | 21.96 | 3.73 | 3.69 | 1049.83 | 6 | 0 | 0 |
| release when swapping ends (not paper default) | 50% | 37.11 | 50.29 | 23.65 | 23.51 | 13222.00 | 6 | 0 | 0 |
| retain until exit (paper default) | 30% | 2.37 | 2.67 | 2.67 | 1.83 | 0.00 | 0 | 0 | 6 |
| retain until exit (paper default) | 40% | 20.81 | 18.69 | 18.69 | 2.97 | 0.00 | 0 | 0 | 6 |

| Token handling | Shortage | Global LRU | Token, random | Token, sync priority | Sync priority + hand-over (extension) | Priority assignments (mean) | Seeds sync better | Seeds sync worse | Seeds identical |
|---|---|---|---|---|---|---|---|---|---|
| retain until exit (paper default) | 50% | 37.11 | 24.88 | 24.88 | 19.40 | 0.00 | 0 | 0 | 6 |

- **Paper-default handling (token kept until its holder exits): the rule is inert.** The token is assigned once, at the first shortage, before any rank is waiting, so sync priority and random assignment were identical in 18 of 18 paired runs. The rule is therefore a refinement for systems where the token is re-assigned, not a general improvement.
- **Token released whenever the holder stops swapping (frequent re-assignment): large gains.** The sync rule beat random assignment in 18 of 18 paired runs. Random assignment is itself harmful there (many token transfers, the effect the paper warns about); sync priority brings the application back near the global-LRU level at 30% shortage, and well below it at 40% (see table).
- **Case-2 forced releases: mixed.** It helps in most runs but is clearly worse than random in some (for example 40% shortage with the 250k threshold), so no uniform improvement is claimed.
- **Extension (not in the paper):** handing the token to the waited-on process helps in many settings (e.g. 40% shortage under paper-default handling) but not all (e.g. 50% shortage with Case-2 release).

*Caveats:* one scenario, six seeds, synthetic traces, and a single-node stand-in for inter-node synchronisation; the benefit of the rule in a real cluster depends on factors not modelled here.

## 7. Comparison with the published results

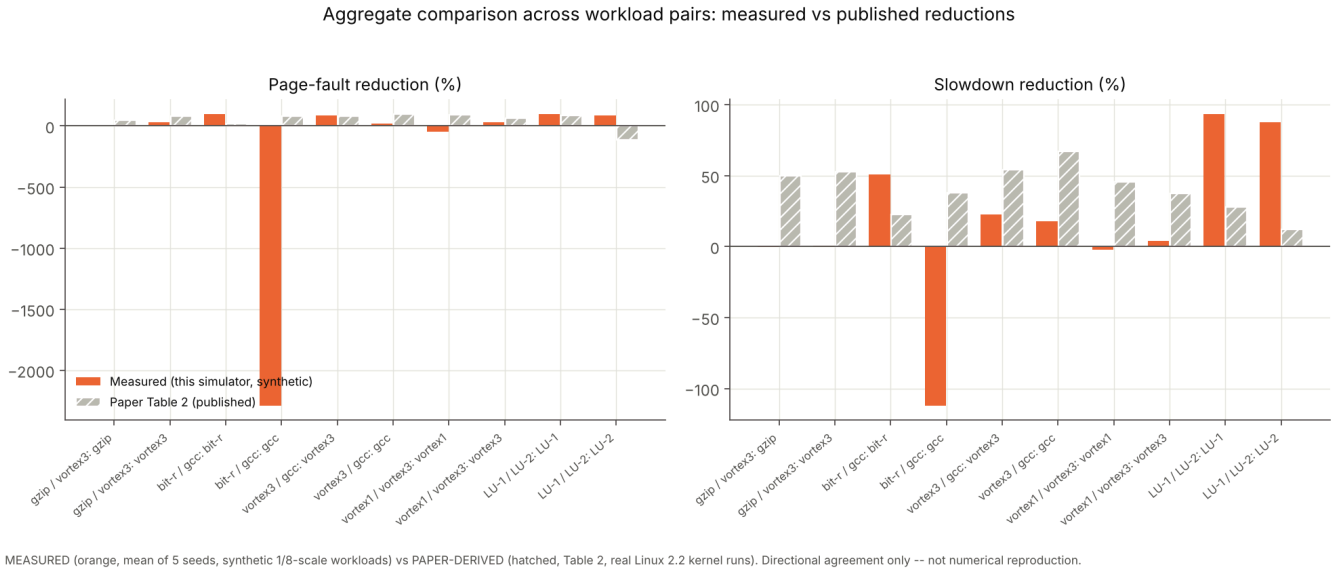



Figure 9 (measured vs paper-derived): reductions for each program. Orange = this simulator; hatched = paper Table 2.

| Pair | Program | Paper faults LRU→token | Ours faults LRU→token | Paper slowdown | Ours slowdown | Direction (faults / slowdown) |
|---|---|---|---|---|---|---|
| gzip_vortex3 | gzip | 328,551 → 179,908 (+45%) | 3,938 → 3,938 (+0%) | 5.23 → 2.63 (+50%) | 2.43 → 2.44 (-1%) | NO / NO |
| gzip_vortex3 | vortex3 | 294,038 → 59,945 (+80%) | 229 → 158 (+31%) | 3.85 → 1.83 (+52%) | 1.15 → 1.14 (+1%) | yes / yes |
| bitr_gcc | bit-r | 108,352 → 86,467 (+20%) | 5,310 → 83 (+98%) | 2.69 → 2.08 (+23%) | 2.57 → 1.27 (+51%) | yes / yes |
| bitr_gcc | gcc | 139,807 → 25,406 (+82%) | 210 → 5,021 (-2289%) | 3.63 → 2.25 (+38%) | 1.45 → 3.07 (-112%) | NO / NO |

| Pair | Program | Paper faults LRU→token | Ours faults LRU→token | Paper slowdown | Ours slowdown | Direction (faults / slowdown) |
|---|---|---|---|---|---|---|
| vortex3_gcc | vortex3 | 248,045 → 51,749 (+79%) | 1,102 → 172 (+84%) | 3.37 → 1.54 (+54%) | 1.62 → 1.25 (+22%) | yes / yes |
| vortex3_gcc | gcc | 275,982 → 12,425 (+96%) | 3,129 → 2,576 (+18%) | 5.61 → 1.85 (+67%) | 2.77 → 2.28 (+18%) | yes / yes |
| vortex1_vortex3 | vortex1 | 193,399 → 13,842 (+93%) | 649 → 947 (-46%) | 3.58 → 1.95 (+46%) | 2.03 → 2.08 (-2%) | NO / NO |
| vortex1_vortex3 | vortex3 | 208,342 → 76,171 (+63%) | 1,015 → 708 (+30%) | 3.33 → 2.08 (+38%) | 1.96 → 1.88 (+4%) | yes / yes |
| lu1_lu2 | LU-1 | 138,372 → 17,999 (+87%) | 25,190 → 1,063 (+96%) | 3.57 → 2.57 (+28%) | 42.41 → 2.79 (+93%) | yes / yes |
| lu1_lu2 | LU-2 | 77,680 → 168,215 (-116%) | 25,190 → 4,205 (+83%) | 3.40 → 2.99 (+12%) | 42.41 → 5.35 (+87%) | NO / yes |

| Metric | Programs | Same sign as paper | Mean \|difference\| (pct. points) |
|---|---|---|---|
| major page faults | 10 | 6 | 301 |
| slowdown | 10 | 7 | 58 |

**Reading this honestly.** Directional agreement is partial and numerical agreement is not claimed. The simulator differs from the original experiment in: synthetic traces at 1/8 scale; exact timestamp LRU instead of Linux’s reference-bit approximation; a single fixed-time disk; no write-back, swap cache or read-ahead; our own choice of shortage per pair and of the monitor thresholds. Several results depend on which process receives the token first: in bit-r/gcc the token went to bit-r in all five seeds, which helps bit-r and hurts gcc, whereas the paper reports gcc benefiting strongly. We did not adjust parameters to force a match.

## 8. Validation

109 automated tests cover the 15 areas of the brief (known LRU sequence, token acquisition only under shortage, protection, restoration of eligibility, fallback, release on completion/timeout/exit, unsuitable marking and cooldown, single owner, empty traces, tiny memory, termination while holding the token, determinism, metric formulas) plus the free/unmap operation and the synchronisation groups, barriers and token priority (mutation checks confirm that disabling the priority or inverting it fails 26 tests). Deliberately breaking the protection, the fallback or the shortage check made tests fail (mutation checks). Structural invariants (frames never over-committed, no duplicate resident pages, token never held by a finished process) were asserted on all five pairs at two pressure levels under both policies. A clean rerun of the experiments reproduced the CSV outputs exactly.

```
$ python -m pytest -q
109 passed in 0.31s

$ python experiments/demo.py
tiny demo (300 + 340 pages, 400 frames)  (synthetic; simulated ticks)

== global_lru
  A*   major faults  10206  elapsed  2045224  slowdown 145.78
  B*   major faults  10284  elapsed  2054866  slowdown 129.32

== token_lru
  A*   major faults    102  elapsed    34528  slowdown   2.46
  B*   major faults    466  elapsed    72484  slowdown   4.56
  token: {'token_acquisitions': 1, 'protected_skips': 244, 'fallback_evictions': 0} events: [(600,
'acquire', 0, ''), (34528, 'release_exit', 0, '')]
```

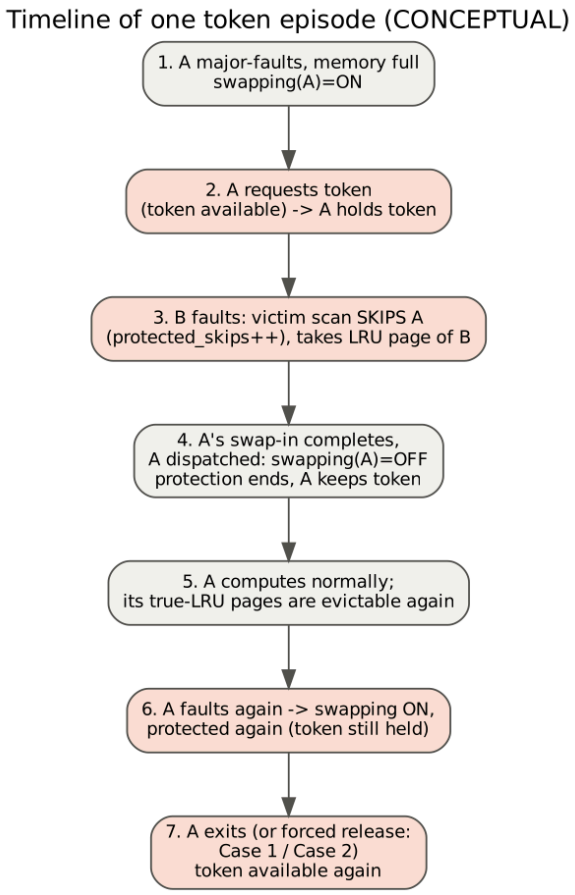



Figure C (conceptual illustration): the life of one token episode.

## 9. Limitations and threats to validity

- Simulator, not a kernel patch; none of the paper's Linux 2.2.14 code was built or run.
- Workloads are synthetic and scaled; they follow the paper's tables and figures but are not SPEC2000/bit-r/LU executions. The paper's plotted bit-r MAD (≈65.5k pages) disagrees with its Table 1 (131.3 MB ≈ 33.6k pages); we followed Table 1 and the RSS curve.
- Synchronisation-aware priority is modelled only for barrier-synchronised processes on one node. Real parallel systems wait on remote nodes and messages, which are not modelled; the evaluation scenario is our own design.
- Baseline pathologies (exact LRU on cyclic scans: LU and Section 5.4) inflate baseline slowdowns relative to Linux; reduction percentages there overstate what a Linux baseline would show.
- Five seeds; random assignment among symmetric processes makes some pairs seed-dependent (e.g. vortex1/vortex3: token-better in 3 of 5 seeds for vortex1).

## 10. Conclusion

The simulator implements every element of the paper's token-ordered LRU algorithm, including a single-node model of the synchronisation-aware priority, and demonstrates the paper’s central mechanism: protecting a swapping token holder sharply reduces false-LRU evictions and, in several workload pairs and in the Section 5.4 example, large reductions in faults and slowdown follow. The effect is workload- and pressure-dependent, with neutral and negative cases that the paper’s five-pair table does not show; verifying those against a real kernel is outside what a simulator can establish.

## Appendix: reproduce

```
pip install -r requirements.txt
python -m pytest -q
python experiments/run_all.py --seeds 5
python experiments/summarize.py
python experiments/make_plots.py
python experiments/run_sync.py --seeds 6    # synchronisation-aware priority experiment
python experiments/write_report.py
python experiments/build_report.py
```